실지출 트래커 개발 인수인계 (PPT팀 전달용)

Nov 4, 2026 · @coldsol

한 장 요약

토스뱅크 거래내역서를 올리면 친구들이 보내 준 더치페이와 결제 취소를 빼고 **내가 실제로 쓴 돈**을 보여주는 웹앱을 만들었어요. 분류와 주간 보고서는 Claude Console에서 만든 **전용 에이전트 2개**가 나눠 맡아요.

발표에서 강조할 메시지 3개:

1. **토스 내역 ≠ 실제로 쓴 돈.** 18만원 술값도 친구 3명이 4.8만원씩 보내 주면 내 몫은 4.2만원이에요. 이 앱은 그 차이를 자동으로 찾아서 빼 줘요.
2. **역할을 나눈 AI 에이전트, 숫자로 증명.** 직접 만든 정답 데이터 569건으로 채점해서, 분류 에이전트 v8이 69%로 단일 Haiku 모델(63%)과 첫 에이전트(55%)를 이겼어요.
3. **소비 → 행동으로.** 주간 보고서가 표·그래프로 줄일 곳과 다음 주 목표를 짚고, 소비와 이어지는 청년 정책(교통비 → K-패스)까지 추천해요.

기획 의도

대학생의 소비 기록은 은행 앱에 다 있지만, 그대로는 "내가 진짜 얼마를 무엇에 썼는지"를 알 수 없어요. 이 앱은 아래 세 가지 문제를 풀려고 만들었어요.

문제	실제 상황	앱의 해결
더치페이	내 카드로 한 번에 내고 친구들이 나중에 송금 → 지출은 부풀려 보이고 입금은 수입처럼 보임	AI가 친구 입금을 원래 결제에 연결 → 확정하면 실지출에서 빼기
이름만으로는 모르는 결제	토스페이·카카오페이·친구 송금·"(주)○○" 법인명 → 카테고리를 알 수 없음	분류 에이전트가 앞뒤 거래·정산·사용자의 예전 선택을 찾아보고 추리
기록만 있고 행동은 없음	한 달 지나야 "많이 썼네"를 알게 됨	매주 보고서가 줄일 곳·다음 주 목표를 제안, 청년정책까지 연결

대상은 토스뱅크를 쓰는 대학생이고, 친구에게 말하듯 해요체로 돕는 톤을 지켰어요. 잔소리보다 "얼마를 아낄 수 있었는지"를 금액으로 보여주는 게 목표예요.

개발한 기능

화면 8개와 AI 기능을 만들었어요. 아래 메뉴는 홈·거래·올리기(+)·하루·보고서, 나머지는 홈에서 들어가요.

화면	하는 일	의도
홈	이번 달 실제로 쓴 돈, 카테고리 원그래프(비율·금액), 오늘·하루 평균·수입, 이번 주 막대, 할 일 카드	앱을 열자마자 "진짜 쓴 돈"과 다음에 할 일을 한 화면에
거래내역 올리기	토스 거래내역서 PDF·엑셀·CSV 업로드, 비밀번호 PDF 지원, 중복 거래 자동 제외	은행 연동 없이 파일 하나로 시작
거래내역	월별 전체 거래, 분류 확인 목록(AI 추측·확신도·근거), 카테고리 바꾸기	AI가 확신 없는 거래만 사용자가 고르게
정산	AI가 찾은 친구 입금 ↔ 결제 연결 확인, 직접 연결(결제 여러 개 묶기)	더치페이를 실지출에서 빼기
하루	그날 쓴 돈, 카테고리, 결제·받은 돈 목록, 하루 요약 알림	매일 저녁 하루를 돌아보기
주간 보고서	한 줄 요약 → 요일별 막대그래프 → 예산 대비표 → 지난주 비교 그래프 → 줄일 수 있었던 곳 → 다음 주 목표. 지난주 보고서는 앱을 열면 자동 작성	기록을 행동으로 바꾸기
청년정책	태어난 해·지역·상태·자취 4가지로 직접 조사한 16개 정책 중 맞는 것 추천, 소비와 이어지는 정책을 맨 앞에	"아끼는 법"까지 알려주는 앱
설정	카테고리별 월 예산, 카테고리 추가(데이트·여행 등 설명을 적으면 AI가 반영), 알림 시간	내 생활에 맞게

카테고리는 13개(식비·편의점·술·유흥·구독료·교통비·쇼핑·생활·데이트·여행·송금·기타·제외)예요. 들어온 돈은 정산·용돈·알바비·환불·기타 수입 5가지로 나눠요.

AI 구성

애매한 분류와 보고서는 전용 에이전트 2개가 맡아요



AI 구성 · 에이전트 2개, Haiku 호출 2개, 규칙 1개

쉬운 일은 Haiku 모델을 한 번 부르고, 추리가 필요한 분류와 보고서는 Console 전용 에이전트가 맡아요. 에이전트가 실패해도 Haiku가 대신해서 앱은 멈추지 않고, 숫자는 항상 앱 코드가 계산해 에이전트에게 넘겨요.

AI가 하는 일

거래내역을 올리면 AI가 아래 순서로 일하고, 사용자는 AI가 확신하지 못한 것만 골라요.

순서	AI	하는 일	예시	사용자가 하는 일
1	Haiku · 파일 읽기	토스 PDF·엑셀의 표가 일정하지 않을 때 내용을 읽어 거래(일시·이름·금액)로 정리	열 이름이 다른 엑셀도 자동으로 맞춤	없음

순서	AI	하는 일	예시	사용자가 하는 일
2	Haiku · 가게 이름 분류	이름만 봐도 알 수 있는 가게를 가게 단위로 한 번에 분류	GS25 → 편의점, 넷플릭스 → 구독료	틀리면 한 번 바꾸면 같은 가게에 모두 적용
3	Haiku · 정산 연결	친구가 보낸 돈이 어느 결제의 더치페이인지 금액·시간·사람 수로 추리해 제안	18만원 술값 다음 날 4.8만원씩 3명 → 그 술값의 정산	정산 화면에서 "맞아요" 누르면 실지출에서 빠짐
4	분류 에이전트 v8	이름만으로 모르는 거래를 거래 하나씩 추리. 앞뒤 거래·친구 정산·사용자의 예전 선택을 도구로 찾아보고, 답마다 확신도와 근거 한 줄을 냄	만화카페 결제 10분 뒤 카카오페이 1만원 → 유흥	확신 낮은 것만 분류 확인에서 고름
5	Haiku · 들어온 돈 분류	입금을 정산·용돈·알바비·환불·기타 수입 5가지로 나눔 (98% 정확)	매달 같은 날 같은 금액 → 알바비	없음
6	보고서 에이전트 v3	매주 앱이 계산한 숫자를 도구로 받아, 늘어난 카테고리의 실제 결제를 찾아보고 표·그래프·절약 제안·다음 주 목표를 씀	"목요일 택시 3번(3.8만원) → 막차 전 지하철로 3.4만원 아낌"	읽고 다음 주 목표 지키기

AI가 하지 않는 일도 있어요. 실지출 합계·비율 같은 숫자는 항상 앱 코드가 계산하고(AI가 숫자를 지어내지 않게), 청년정책 추천은 나이·지역 같은 규칙으로만 거러요. 에이전트 둘은 실패하면 Haiku가 대신해서 앱이 멈추지 않아요.

AI 성능을 올린 과정

처음에는 Haiku 모델 하나로 분류와 보고서를 다 했고, Claude Console에서 분류 특화 에이전트를 만든 뒤 시스템 프롬프트와 스킬을 더해 성능을 올렸어요. 단계마다 직접 만든 정답 데이터로 채점해서, 좋아졌는지를 숫자로 확인했어요.

1. **Haiku 모델 하나로 전부 (시작점, 63~67%)**
   - 거래 목록을 한 번에 넘겨 분류하고, 같은 모델이 주간 보고서도 썼어요.
   - 페이 결제·친구 송금처럼 이름만으로 모르는 거래는 32~38%밖에 못 맞혔어요.
2. **정답 데이터 569건 만들기**
   - 개선이 진짜인지 재려면 기준이 필요해서, 3개월 거래에 클릭 도구로 직접 정답을 달았어요.
3. **Console에서 분류 특화 에이전트 빌드 (55%)**

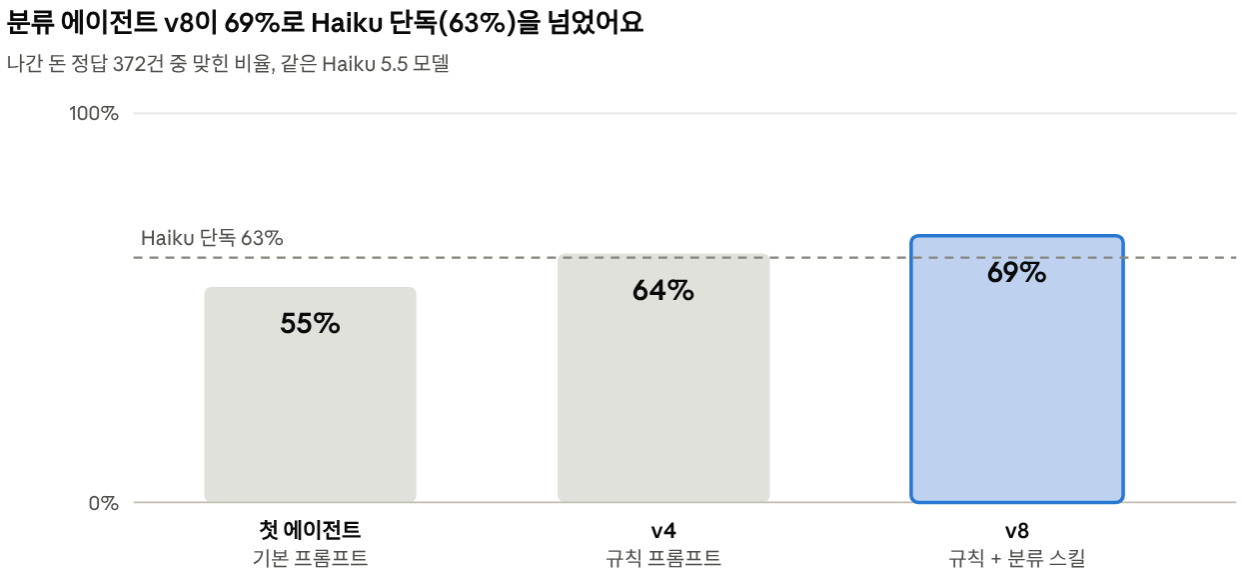
- 에이전트가 앱의 도구 4개(앞뒤 거래·친구 정산·사용자의 예전 선택·거래 찾기)로 스스로 더 찾아보게 했어요.
- 첫 결과는 오히려 낮았어요. 틀린 것을 분석하니 마트를 유흥·기타로, 데이트를 식비로, 친구 송금을 무조건 "송금"으로 고르고 있었어요.

4. **시스템 프롬프트에 규칙 추가 (v4, 64%)**
   - 찾아보는 순서(같은 이름의 정답 → 앞뒤 거래 → 같은 날 패턴)와, 틀린 패턴을 고치는 규칙을 넣었어요.
   - 페이·송금 정확도가 30%에서 45%로 올랐어요.
5. **분류 규칙을 스킬로 분리 (v8, 69%)**
   - 카테고리 13개 정의, 데이트·여행 우선 규칙, 법인명 가게 읽는 법, 예시를 규칙집 스킬(transaction-categorizer)로 만들어 붙였어요.
   - 처음으로 Haiku 단독을 넘었고, 편의점은 64%에서 95%가 됐어요.
6. **보고서도 같은 방식으로 (보고서 에이전트 v3)**
   - Console에 보고서 특화 에이전트를 만들고, 표·그래프 양식과 절약 제안 가이드를 스킬(weekly-report-writer)로 붙였어요.

시스템 프롬프트에는 역할과 일하는 순서를, 스킬에는 자세한 규칙과 예시를 두었어요. 스킬은 버전이 남고 팀원도 고칠 수 있어서, 다음 개선(데이트 분류 등)도 같은 방식으로 하면 돼요.

성능 검증 결과

같은 Haiku 모델에 규칙과 스킬을 더하자 분류 정확도가 55%에서 69%로 올랐어요. 토스 3개월 거래 569건(나간 돈 372, 들어온 돈 197)에 직접 정답을 달고, 세 묶음으로 나눠 시험 묶음의 정답을 가린 채 채점했어요.



직접 만든 정답 데이터 · 나간 돈 372건 · 2026년 11월 3일 채점

항목	첫 에이전트	v4	v8	Haiku 단독
가게 결제 (262건)	66%	72%	82%	76%
페이·송금 (110건)	30%	45%	37%	32%
채점 1회 비용	$0.07	$0.09	$0.06	$0.05

- 가장 크게 오른 건 편의점(64% → 95%)이에요. 스킬에 넣은 "마트·할인점·법인명 가게 읽는 법" 덕분이에요.
- 들어온 돈 종류 분류(정산·용돈·알바비 등)는 197건 중 193건(98%)을 맞혔어요.
- 자세한 수치는 저장소의 `docs/에이전트_v8_성능검증.pdf` 에 있어요.

기술 스택과 구조

내 컴퓨터에서 도는 웹앱이고, 거래 데이터는 밖으로 나가지 않아요(AI 호출 때 필요한 거래만 전달).

영역	쓴 것
서버	Python, FastAPI
저장	SQLite (내 컴퓨터의 파일 하나)
화면	HTML·CSS·JavaScript (프레임워크 없이), Chart.js(그래프), 휴대폰 앱 모양
AI	Claude Haiku 5.5 (API), Claude Console의 Managed Agents 2개(분류·보고서), 스킬 2개
파일 읽기	토스 PDF(비밀번호 포함)·엑셀·CSV 파서
품질	자동 테스트 110개, 정답 데이터 채점 도구
공유	설치 없이 열리는 데모 HTML 한 파일(가짜 거래 데이터)

PPT 장표 제안

개발 파트는 아래 9장이면 충분해요. 장마다 쓸 자료는 이 문서의 표·그림과 데모 화면에서 가져가면 돼요.

#	장 제목	넣을 내용	자료
1	토스에 찍힌 돈 ≠ 내가 쓴 돈	18만원 술값 → 친구 3명 정산 → 내 몫 4.2만원 예시	기획 의도 표

#	장 제목	넣을 내용	자료
2	우리가 푼 문제 3가지	더치페이 / 이름만으로 모르는 결제 / 행동으로 안 이어짐	기획 의도 표
3	서비스 흐름	PDF 올리기 → 분류·정산 → 실지출 → 보고서 → 정책	AI 구성 그림
4	홈 화면	실제로 쓴 돈 + 원그래프	데모 홈 캐처
5	AI 에이전트 역할 분담	분류 에이전트 / 보고서 에이전트 / 스킬 / 실패하면 Haiku가 대신	AI 구성 그림
6	숫자로 증명한 성능	55% → 64% → 69%, Haiku 단독 63%	성능 검증 그래프
7	주간 보고서	그래프·표·줄일 곳·다음 주 목표	데모 보고서 탭 캐처
8	청년정책 추천	내 정보 4가지 → 맞는 정책, 교통비 → K-패스 연결	데모 정책 화면 캐처
9	한계와 다음 단계	데이트 분류 약함, 메모리(쓸수록 똑똑해지는 에이전트), 정책 API 연동	자료 위치·한계

데모를 직접 보여줄 수 있다면 4·7·8장은 라이브 데모로 바꿔도 돼요.

자료 위치와 한계

자료	어디	쓰임새
데모 링크	실지출 트래커 데모	화면 캐처, 라이브 데모 (휴대폰에서도 열림)
데모 파일	개발 담당이 단톡방에 공유한 `실지출트래커_데모.html`	인터넷 없이 PC에서 열기
코드	GitHub coldsol2/spending-tracker	기술 장표의 근거
성능 검증 PDF	저장소의 `docs/에이전트_v8_성능검증.pdf`	성능 장표의 출처

발표에서 솔직하게 말해야 할 점:

- 데모는 **가짜 거래 데이터**로 만든 거예요. 실제 거래내역·친구 이름은 넣지 않았어요.
- 데모의 보고서는 데모 숫자로 계산한 예시예요. 실제 앱에서는 보고서 에이전트가 써요.
- 성능은 **한 사람의 3개월 데이터**로 잰 것이고, v8은 한 번만 채점했어요. Haiku 단독은 돌릴 때마다 63~67%였어요.

- 데이트 분류는 아직 약해요(30건 중 3건). 다음 개선 대상이에요.
- 청년정책은 공공 API 대신 직접 조사한 목록(2026년 11월 4일 기준)이에요. 금액은 출처마다 조금 달라서 발표 전에 공식 누리집으로 한 번 더 확인하면 좋아요.
- 최신 변경(원그래프·청년정책·데모 갱신)은 아직 GitHub에 올리지 않았어요.